Divisão e Conquista

Trabalho prático de Análise de Algoritmos
Universidade de Fortaleza - Ciências da Computação

Vinícius Estevam Silva
Segundo integrante: Davi Laurentino Oliveira de Souza

1. Objetivo e organização

Implementar e comparar força bruta e divisão e conquista em dois problemas: subarranjo de soma máxima e par de pontos mais próximos. A comparação considera correção, complexidade e tempos medidos. Foi acrescentado Kadane, linear, como complemento à menção de um desafio O(n) nos critérios do enunciado.

A implementação foi feita em Python. Os algoritmos ficam separados da interface de entrada, dos testes e dos experimentos. O notebook permite reproduzir a execução no Google Colab; os arquivos .py também podem ser executados localmente.

Convenções e interpretações

Os índices começam em zero. O subarranjo é não vazio e seus extremos são inclusivos. Em empate de soma, escolhe-se o menor índice inicial e, depois, o menor final. Para pontos, aceita-se qualquer par ótimo, identificado pelos índices originais; as distâncias são euclidianas.

O enunciado não fornece o formato exato de um avaliador automático. A interface adotada está documentada no README. A palavra “dígitos” da Etapa 2 foi interpretada como número de pontos; o limiar foi interpretado como tamanho do caso-base recursivo.

Proveniência: os resultados deste PDF foram medidos em ambiente remoto, identificado na seção experimental. Não representam medições no computador pessoal dos integrantes. O script gerar_relatorio.py permite atualizar o documento após uma nova execução.

2. Subarranjo de soma máxima

Força bruta: para cada início i, percorrem-se os finais j e atualiza-se uma soma acumulada. São examinados n(n+1)/2 intervalos. A soma não é recalculada do zero, evitando uma terceira repetição.

Uma recorrência equivalente para a contagem do trabalho é $B(n) = B(n-1) + \Theta(n)$, com $B(1) = \Theta(1)$. Expandindo: $B(n) = \Theta(1 + 2 + \ldots + n) = \Theta(n^2)$. O espaço auxiliar é $\Theta(1)$, desconsiderando a entrada.

Divisão e conquista: divide-se o intervalo em duas metades. A solução está inteiramente à esquerda, inteiramente à direita ou cruza o meio. O terceiro caso é a soma do melhor sufixo da metade esquerda com o melhor prefixo da direita. A combinação custa $\Theta(n)$.

A recorrência é $D(n) = D(\lfloor n/2 \rfloor) + D(\lceil n/2 \rceil) + \Theta(n)$, com $D(1) = \Theta(1)$. Para potências de dois: $D(n) = 2D(n/2) + \Theta(n)$. Existem $\Theta(\log n)$ níveis, cada um com trabalho total $\Theta(n)$; logo $D(n) = \Theta(n \log n)$. A pilha recursiva ocupa $\Theta(\log n)$.

Correção: os três casos cobrem todos os intervalos não vazios. Por indução, cada chamada fornece a solução ótima da sua metade. O melhor intervalo cruzado combina independentemente o maior sufixo e o maior prefixo. Escolher o melhor dos três preserva a otimalidade. O comparador também aplica a regra de desempate.

Complemento: Kadane

Mantém-se a melhor soma terminando na posição atual. Se a soma anterior é negativa, inicia-se um novo intervalo; caso contrário, estende-se o existente. Quando a soma anterior é zero, estender preserva o menor início no desempate. A resposta global é atualizada a cada elemento.

$K(n) = K(n-1) + \Theta(1)$, portanto $K(n) = \Theta(n)$, com espaço $\Theta(1)$. O estado inicial utiliza o primeiro elemento, não zero; assim, vetores inteiramente negativos retornam o maior elemento, e não um intervalo vazio.

Exemplo: para [-2, 3, -1, 4, -5], as três versões retornam soma 6, início 1 e fim 3. Para [-8, -3, -6], retornam soma -3 e extremos 1 e 1.

3. Par de pontos mais próximos

Força bruta: são comparados todos os n(n-1)/2 pares. A recorrência de contagem é B(n) = B(n-1) + Θ(n), com B(2) = Θ(1), resultando em Θ(n²). O espaço auxiliar é Θ(1). Usa-se hypot para calcular a distância euclidiana.

Divisão e conquista: os pontos são inicialmente ordenados por x, com y e identificador como desempate. A lista é dividida por posição, garantindo metades equilibradas mesmo quando vários pontos têm o mesmo x. Cada chamada retorna também seus pontos ordenados por y; as duas listas são intercaladas em tempo linear.

Seja δ a menor distância dos dois lados. Um par entre metades que melhore δ deve estar na faixa de largura 2δ em torno da divisão. A faixa é construída a partir da lista já ordenada por y. Cada ponto é comparado com no máximo os sete seguintes, interrompendo quando a diferença em y já é pelo menos a melhor distância.

Por que bastam sete vizinhos?

Para um ponto p, só interessam sucessores na faixa com diferença vertical menor que δ. Eles ficam num retângulo de largura 2δ e altura δ. Separando pelas duas metades recursivas, cada lado cabe num retângulo δ por δ, dividido em quatro células de lado δ/2. Dois pontos do mesmo lado na mesma célula teriam distância no máximo δ/√2, menor que δ, contradizendo a distância mínima daquele lado.

Há, portanto, no máximo oito pontos nesse retângulo, contando p: no máximo sete sucessores precisam ser examinados. Pontos com x igual ao corte continuam associados à sua metade recursiva. Fronteiras podem ser atribuídas de modo consistente às células; a busca exige melhora estrita, pois qualquer par empatado é aceito.

Complexidade: com limiar constante L, C(n) = 2C(n/2) + Θ(n) para n > L, e C(n) = Θ(n²) para n ≤ L. Como L não cresce com n, o caso-base é limitado por uma constante. A ordenação inicial custa Θ(n log n) e a recorrência também; o total é Θ(n log n), com espaço auxiliar de pico O(n).

Correção e duplicatas: a melhor resposta ou está em uma metade ou aparece na faixa. A indução e a propriedade dos sete vizinhos cobrem ambos os casos. Coordenadas repetidas são detectadas após a ordenação inicial e retornam distância zero, que é ótima. Essa entrada pode terminar mais cedo; Θ(n log n) descreve o pior caso.

4. Testes e metodologia experimental

Resultado da bateria: 4825 verificações concluídas sem falhas.

Grupo	Cobertura e justificativa
9 casos fixos de soma	Negativos, unitários, zeros, empates, cruzamento, positivos e inteiros grandes.
8 casos fixos de pontos	Dois pontos, duplicatas, colinearidade, x iguais, negativos, frações, empates e par entre metades.
1.092 vetores exaustivos	Todos os vetores de tamanho 1 a 6 sobre {-1, 0, 1}. Oráculo independente enumera intervalos com soma direta.
300 conjuntos aleatórios	2 a 100 pontos; comparação com força bruta em limiares 2, 3, 8, 16 e 32. Validação dos IDs e da distância do par retornado.
Entradas inválidas	Vetor vazio, menos de dois pontos e coordenada não finita devem gerar ValueError.

As verificações fixas incluem saídas esperadas em casos_teste.json. Pontos usam tolerância de 10^{-10} nos testes. A validação automática reduz o risco de erro, mas não substitui a prova nem garante compatibilidade com um formato externo ainda desconhecido.

Foram usados n = 100, 200, 400, 800, 1.600, 3.200 e 6.400, com cinco execuções por tamanho e algoritmo. Os vetores usam inteiros uniformes de -100 a 100; os pontos usam coordenadas uniformes em [-10.000, 10.000]. Em cada repetição, os algoritmos do mesmo problema recebem a mesma entrada. Cada repetição usa uma semente diferente e reproduzível.

O cronômetro perf_counter_ns mede somente a chamada do algoritmo, incluindo a ordenação inicial na solução de pontos. Geração, impressão e validação ficam fora. Há aquecimento curto com até 32 elementos e ordem de execução embaralhada. Foram registradas 175 medições principais; o estudo de limiares acrescenta 30. O resumo contém média, desvio amostral, mínimo e máximo.

Ambiente: Python 3.13.18; Linux-6.6.159+-x86_64-with-glibc2.39. Processador visível: Intel(R) Xeon(R) CPU @ 2.20GHz. CPUs lógicas visíveis: 2. MemTotal: 13286932 kB. Execução UTC: 2026-11-05 22:07:33.

O ambiente é virtualizado e compartilhado; os recursos visíveis não garantem recursos exclusivos. Escalonamento, alocações e coleta de lixo podem influenciar os tempos. Não foi feito isolamento de CPU. Os experimentos principais não cobrem todas as distribuições de entrada.

5. Resultados: subarranjo de soma máxima

n	Força bruta (ms)	Divisão (ms)	Kadane (ms)
100	0,4036	0,2412	0,0615
200	1,6527	0,5283	0,1356
400	6,9883	1,1027	0,2460
800	31,9635	2,6753	0,6249
1600	121,0412	4,8319	1,0049
3200	597,7283	13,2670	2,8739
6400	2179,7294	21,2868	4,8517

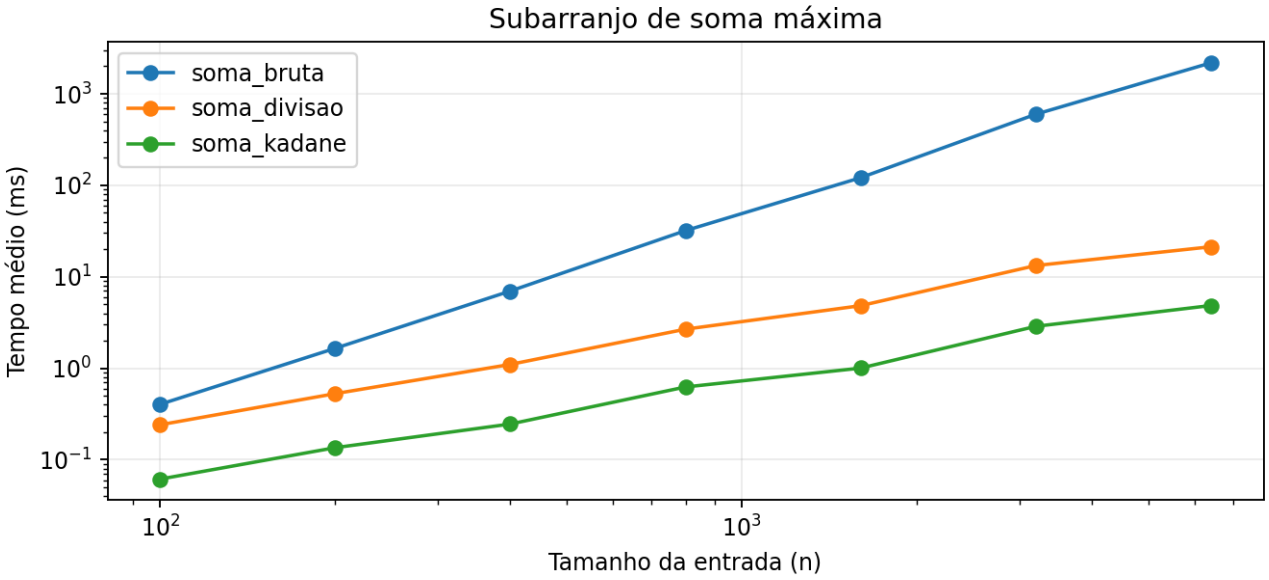


Figura 1. Médias de cinco execuções, em escalas logarítmicas nos dois eixos. O CSV permite consultar a dispersão; as linhas conectam apenas os tamanhos efetivamente medidos.

Ao dobrar n de 3.200 para 6.400, os fatores observados foram 3.65 na força bruta, 1.60 na divisão e 1.69 em Kadane. Para n^2, $n \log_2 n$ e n, os fatores teóricos são 4, aproximadamente 2,17 e 2. Os fatores medidos não precisam coincidir exatamente com essas referências, devido às constantes e à variabilidade.

Em n = 6.400, a força bruta levou 102.4 vezes o tempo da divisão. O crescimento observado é compatível com a vantagem assintótica de divisão e conquista; Kadane oferece a alternativa linear para este problema específico. Tempos finitos não constituem prova de complexidade.

6. Resultados: par de pontos mais próximos

n	Força bruta (ms)	Divisão (ms)
100	1,0997	0,4996
200	4,2930	1,0199
400	17,7223	2,1546
800	77,0555	4,9993
1600	294,7987	9,7972
3200	1414,5146	29,9446
6400	5636,0555	50,8237

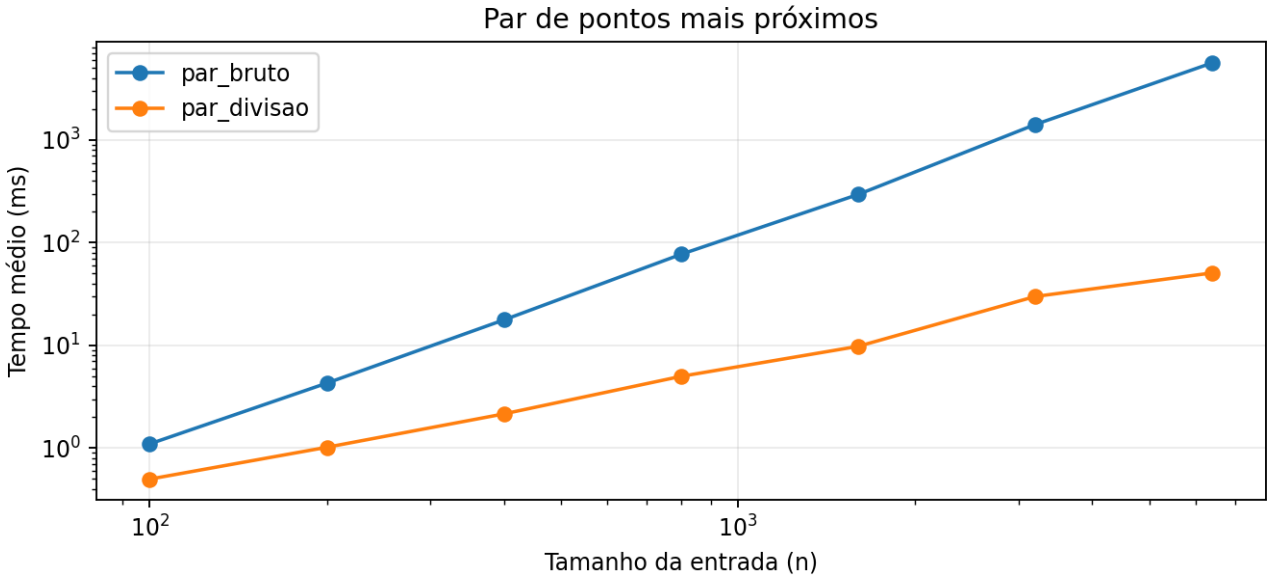


Figura 2. Médias de cinco execuções. A ordenação inicial faz parte dos tempos de divisão e conquista. O limiar utilizado nas medições principais foi 16.

Entre 3.200 e 6.400 pontos, a força bruta cresceu por fator 3.98, enquanto a divisão cresceu por 1.70. As referências teóricas são 4 e aproximadamente 2,17, respectivamente. Em 6.400 pontos, a razão entre os tempos médios foi 110.9 a favor da divisão.

A divisão foi mais rápida nos seguintes tamanhos medidos: 100, 200, 400, 800, 1600, 3200, 6400. Em entradas muito pequenas, a força bruta pode economizar ordenação, recursão e alocação de listas. Este ensaio começa em n = 100 e não determina o ponto exato de cruzamento nem comprova uma vitória da força bruta abaixo desse valor.

7. Limiar, limitações e conclusão

Limiar	Média em 3.200 pontos (ms)
2	26,1278
3	20,9553
8	19,8862
16	23,0738
32	28,3437
64	42,6474

O limiar 16 foi definido antes dos experimentos principais como compromisso entre reduzir chamadas recursivas e manter pequenos os grupos quadráticos. Para avaliá-lo, foram feitas cinco execuções com cada candidato, reutilizando os mesmos cinco conjuntos de pontos e alternando a ordem dos candidatos. As sementes e os tempos estão em limiares.csv.

O menor tempo médio desta avaliação foi obtido com limiar 8. O limiar 16 levou 1.16 vezes esse tempo. Portanto, 16 não é apresentado como o parâmetro ótimo universal. Foi mantido para preservar a decisão inicial e a consistência dos resultados principais. Uma otimização posterior pode adotar o vencedor e repetir todo o experimento; alterar L para outra constante não altera a ordem assintótica.

A força bruta é simples e útil como referência de correção e caso-base. A divisão reduz fortemente o crescimento do custo para entradas maiores. No subarranjo, Kadane melhora ainda mais a ordem, mostrando que a melhor técnica depende da estrutura do problema.

Limitações: apenas cinco repetições por tamanho, uma família aleatória principal por problema e ambiente compartilhado. Não há afirmação de significância estatística. A aritmética foi analisada sob custo unitário; inteiros Python têm precisão arbitrária e seu custo cresce com o número de bits. Para pontos, valores extremos podem sofrer limitações da representação de ponto flutuante.

Reprodutibilidade e referência

Executar, na pasta do projeto: python testes.py; python experimentos.py; python gerar_relatorio.py. Os arquivos de entrada, código, CSV, gráficos e identificação da máquina devem acompanhar o repositório. O histórico Git deve registrar o desenvolvimento real; este pacote não representa histórico prévio de semanas.

Fonte dos requisitos: enunciado fornecido na disciplina, “Trabalho Prático: Divisão e Conquista”, entrega indicada em 28/10/2026. As recorrências, demonstrações e análises acima foram desenvolvidas a partir das implementações e das medições deste projeto.